

Table des matières

Exercices 07 — CI/CD et qualité analytique.....	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	3
Mises en situation	4

Exercices 07 — CI/CD et qualité analytique

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur un univers Boréal redessiné. Ce cours porte encore l'ancienne pile et l'ancienne numérotation : il sert de matière au futur Cours 08, qui n'est pas encore écrit. Les cours 01 à 06 du dossier, eux, sont à jour. Voir `README.md`.

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les énoncés portent sur la Théorie 07 et sur le Labo 07 ; ceux marqués [labo] supposent que vous avez fait le labo.

Le corrigé est dans `Corrige_07.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Branche ou pas. Vous êtes seul sur Boréal. Pourquoi travailler quand même par branche et *pull request* ?
2. La revue utile. Un collègue propose un modèle dont le SQL est impeccable, mais qui n'a ni description ni test. Que demandez-vous, et pourquoi est-ce plus important que la syntaxe ?
3. CI contre CD. Une équipe dit « on fait du CI/CD » et déploie en production à chaque fusion. Est-ce nécessairement un progrès pour une plateforme analytique ?
4. Le *runner* neuf. Votre workflow échoue sur dbt : `command not found`, alors que la commande marche sur votre poste. Que s'est-il passé ?
5. Deux gestes. Pourquoi séparer `sqlfluff` et `dbt build` en deux étapes, au lieu de tout lancer d'un coup ?
6. Le schéma de la CI. Que se passerait-il si la CI construisait dans `BOREAL.MARTS` ?

7. La macro, encore. Comment la CI obtient-elle des schémas `CI_STAGING` et `CI_MARTS` sans qu'on écrive une seule ligne nouvelle dans les modèles ?
8. Le fichier qu'on versionne. Le Cours 00 disait de ne jamais versionner `profiles.yml`. Le Cours 07 en versionne un. Expliquez pourquoi il n'y a pas contradiction.
9. Le secret effacé. Vous avez poussé un mot de passe par erreur, puis fait un commit qui le retire. Le problème est-il réglé ?
10. La table figée. Tous vos tests passent, et pourtant le rapport affiche des chiffres de la semaine dernière. Quel contrôle manquait ?
11. Le test qui crie. Un test signale 40 lignes fautives chaque matin, toujours les mêmes, sans gravité. Que faites-vous, et que ne faites-vous pas ?
12. Le contrat. À quoi sert de déclarer un contrat sur un modèle, alors qu'on a déjà des tests ?
13. Ce que les tests ne voient pas. Donnez un exemple de plateforme où tous les tests passent et où les chiffres sont pourtant faux pour le métier.
14. `manifest.json`. Pourquoi ce fichier rend-il un projet dbt plus exploitable par un agent d'IA qu'une base de données ordinaire ?
15. La limite de l'agent. Pourquoi un agent d'IA est-il moins utile sur un projet mal tenu, alors que c'est justement là qu'on aurait besoin d'aide ?
16. [labo] Le README. Citez trois choses qu'un lecteur technique cherche dans le README d'un projet analytique, et dites laquelle manque le plus souvent.
17. [labo] Le générateur dans le dépôt. Quel est l'argument pour copier `generer_sources.py` dans `Projet_Boreal`, et quel est l'argument contre ?

Questions pratiques

Le cours du déploiement. On écrit du YAML de workflow, de la configuration, et quelques commandes Git. Écrivez-les de mémoire : un fichier de CI qu'on recopie sans le comprendre est exactement ce qui laisse passer un secret.

P1. Le workflow de CI. Écrivez `.github/workflows/ci.yml`. Il se déclenche à l'ouverture d'une proposition de changement vers la branche principale, installe Python et dbt, installe les paquets, puis construit et teste le projet sur la cible `ci`. Les identifiants viennent des secrets du dépôt. Expliquez en une phrase pourquoi le fichier est si verbeux.

P2. Le profil de la CI. Écrivez `ci/profiles.yml`. Il doit pouvoir être versionné sans danger. Dites, en deux lignes, en quoi il diffère de celui de votre poste, et pourquoi cette différence lève la contradiction apparente avec la règle du Cours 00.

P3. Le style, une fois pour toutes. Écrivez le fichier `.sqlfluff` qui configure le dialecte Snowflake et le gabarit dbt. Donnez la commande qui signale les écarts et celle qui en corrige la plupart. Dites ce que cet outil retire des revues de code, et ce qu'il ne retire pas.

P4. La fraîcheur des sources. Complétez la déclaration d'une source pour que dbt surveille son âge : la colonne technique qui porte l'horodatage de chargement, un avertissement au-delà de 36 heures, une erreur au-delà de 72. Donnez la commande. Puis dites pourquoi ce contrôle attrape quelque chose qu'aucun test `not_null` ne verra jamais.

P5. La sévérité graduée. Écrivez le test qui vérifie le format du code postal d'un concessionnaire, en avertissement à partir de 5 lignes fautives et en erreur à partir de 50. Écrivez ensuite un test qui ne doit jamais être gradué, et dites pourquoi.

P6. Le contrat. Écrivez la déclaration qui impose un contrat sur `fct_ventes` : les colonnes, leurs types, et les contraintes qui ont du sens ici. Dites ce qui se passe désormais si quelqu'un renomme une colonne, et en quoi c'est différent du comportement d'avant.

P7. Relire une proposition de changement. Un collègue propose ce modèle, `marts/dim_fournisseur.sql`, sans autre fichier :

```
select
  f.lifnr as fournisseur_id,
  f.name1 as nom,
  p.bezei as pays
from BOREAL.STAGING.STG_LFA1 f
left join BOREAL.STAGING.STG_T005 p on p.land1 = f.land1
```

Listez tout ce que vous signalez en revue, en distinguant ce qui bloque la fusion de ce qui est une remarque. Il y a au moins cinq points.

P8. (avancé) Le secret parti en public. Un mot de passe Snowflake a été committé il y a trois commits, puis retiré au commit suivant, et tout a été poussé sur un dépôt public. Donnez la suite de gestes, dans l'ordre exact, en distinguant ce qui est urgent de ce qui est du nettoyage. Puis écrivez le sélecteur qui ferait de cette CI une CI mince, et dites ce qu'il exige en retour.

Diagnostics

E1. La CI est en place et tourne à chaque proposition de changement. Un matin, le rapport de direction affiche une étoile à moitié construite, alors qu'aucun changement n'a été fusionné. En regardant le workflow, on trouve `dbt build --target prod`. Expliquez ce qui s'est passé, dites pourquoi personne ne l'avait vu jusque-là, et donnez le correctif complet, qui ne tient pas en un seul mot changé.

E2. Un développeur committe un `profiles.yml` avec son mot de passe, s'en aperçoit, le supprime et committe à nouveau. Il annonce sur le canal d'équipe que « c'est réglé, le fichier n'est plus là ». Le dépôt est public depuis deux jours. Expliquez pourquoi ce n'est pas réglé, ce qui est réellement exposé, et ce qu'il faut faire. Dites aussi ce qui, dans le processus, aurait dû empêcher ça, et pourquoi la vérification après coup arrive toujours trop tard.

E3. Une équipe a 140 tests. Une trentaine sont rouges en permanence, sur des adresses mal formatées et des courriels manquants. Pour retrouver un build vert, l'équipe passe tous les tests en `severity: warn`. Six semaines plus tard, un doublon dans une dimension passe en production et double le chiffre d'affaires d'une région pendant douze jours. Reconstituez l'enchaînement, dites ce qu'il fallait faire des trente tests bruyants, et énoncez le principe qui distingue un test qu'on gradue d'un test qui doit bloquer.

Questions à développer

Un court paragraphe de raisonnement attendu. Rédigez avant de lire le corrigé étoffé.

A. La CI qu'on désactive. Une équipe a monté une CI complète, puis l'a progressivement contournée : fusions forcées, tests mis en warn, workflow désactivé « le temps de livrer ». Expliquez le mécanisme qui mène là, et décrivez une CI conçue pour ne pas subir ce sort.

B. Le projet en entrevue. On vous demande de présenter Boréal en cinq minutes à un responsable technique. Quel plan suivez-vous, quels chiffres montrez-vous, et quelle faiblesse assumez-vous d'avance ?

Mises en situation

M1. Imprimerie Sorel, concevoir la CI. Une imprimerie a une équipe de quatre personnes sur un projet dbt de 180 modèles. Le dbt build complet prend 22 minutes et consomme des crédits Snowflake. Les propositions de changement arrivent au rythme de six ou sept par semaine, et touchent le plus souvent un ou deux modèles. L'équipe veut une CI, sans que la facture explose ni que l'attente décourage.

Concevez le dispositif : que lance-t-on à chaque proposition, dans quel ordre, qu'est-ce qui bloque la fusion et qu'est-ce qui se contente d'avertir. Dites où écrit la CI et pourquoi jamais ailleurs. Traitez la question du coût : quelle option existe pour ne pas tout reconstruire, comment elle fonctionne, et ce qu'il faut mettre en place avant de pouvoir s'en servir. Terminez en disant si vous ajoutez du déploiement continu, et défendez votre réponse.

M2. Éoliennes Matane, la revue de code. Vous relisez une proposition de changement qui ajoute l'analyse des arrêts de turbine. Elle contient : un modèle `stg_arret.sql` qui joint la table des arrêts et celle des turbines pour récupérer le parc éolien ; un modèle `fct_arret.sql` qui lit `stg_arret` et agrège les arrêts par turbine et par jour ; aucun fichier `.yaml` ; un message de commit « ajout arrêts » ; et, dans `fct_arret.sql`, un filtre `where motif <> 'MAINTENANCE_PLAN'` sans commentaire.

Rédigez votre revue. Pour chaque point, dites ce que vous demandez et pourquoi, et classez en bloquant ou non. Deux points ne sont pas des fautes de SQL mais des décisions de modélisation qu'il faut faire expliciter : identifiez-les. Terminez en formulant la question que vous poserez à l'auteur au sujet du filtre, car c'est celle qui compte le plus, et expliquez ce qui vous inquiète.

M3. Coopérative Saint-Lambert, la qualité au-delà des tests. Une coopérative a une plateforme qui tourne depuis un an, avec 90 tests dbt tous au vert. Trois incidents sont survenus quand même. En janvier, le fichier de ventes d'un magasin n'est plus arrivé pendant onze jours, sans que rien ne le signale. En mars, un développeur a renommé une colonne d'un mart, ce qui a cassé deux rapports Power BI et un script d'export du service comptable. En juin, la direction a découvert que la « marge » du tableau de bord n'était pas celle que la comptabilité appelle marge, l'écart durait depuis la mise en service.

Pour chacun des trois, dites quel dispositif l'aurait évité, comment il se met en place concrètement, et pourquoi les 90 tests existants ne pouvaient rien y faire. Un des trois n'a aucune solution technique : nommez-le, et dites ce qui le prévient réellement. Terminez en classant les trois par ordre de gravité, avec votre critère.

M4. (avancé) Gestion Portneuf, l'agent d'IA. La direction d'une société de gestion immobilière revient d'une conférence et demande « un agent d'IA qui répond aux questions des gestionnaires sur nos données ». L'état des lieux : l'entrepôt Snowflake existe, alimenté par dbt, 60 modèles. La moitié des modèles n'a pas de `description`. Aucun grain n'est déclaré autrement que dans la tête de deux personnes. Il y a 12 tests, tous des `not_null`. Trois mesures clés (le taux d'occupation, le revenu net, le coût par logement) sont définies différemment selon le rapport qu'on regarde.

Rédigez la réponse que vous donnez à la direction. Elle doit contenir : ce qui rend une plateforme dbt effectivement hospitalière à un agent et pourquoi, ce qui manque ici et dans quel ordre le combler, ce que l'agent pourra faire de crédible une fois le travail fait, et ce qui restera humain. Dites explicitement ce qui arriverait si on branchait l'agent aujourd'hui, en étant précis sur le mode d'échec, car il n'est pas celui qu'on imagine. Terminez par la phrase que vous diriez en entrevue si on vous demandait de résumer votre position en trente secondes.